

AI and Machine Learning Foundations

Artificial Intelligence, or **AI**, is the broad field of making computers perform tasks that normally require human intelligence. **Machine Learning**, or ML, is one way to build AI systems, while **Deep Learning** is a specialized type of machine learning that uses neural networks with many layers.

A simple way to remember the relationship is:

```
Artificial Intelligence
├─ Machine Learning
│   └─ Deep Learning
│       └─ Deep Neural Networks
```

1. What Is Artificial Intelligence?

Artificial Intelligence is the ability of a computer system to perform intelligent tasks.

These tasks may include:

- Understanding human language.
- Recognizing objects in images.
- Making decisions.
- Solving problems.
- Planning routes.
- Recommending movies or products.
- Generating text, images, music, or code.

For example, when a navigation application finds the fastest route, it is using AI techniques to analyze roads, traffic, distance, and travel time.

AI does not necessarily mean that a machine thinks exactly like a human. It means that the machine can perform a task in a way that appears intelligent.

Examples of AI

- A chess-playing computer.
- A chatbot.
- A face-recognition system.
- A voice assistant.
- A fraud-detection system.
- A self-driving vehicle.
- A medical image-analysis system.

Important point

AI is the **largest category**. It includes many different approaches:

- Rule-based systems.
- Search and planning algorithms.
- Expert systems.
- Machine learning.
- Deep learning.
- Natural language processing.
- Computer vision.
- Robotics.

Not every AI system learns from data. Some older AI systems use rules written directly by humans.

```
IF temperature is above 38°C  
THEN display "fever"
```

This is an AI-like rule-based system, but it is not necessarily machine learning.

2. What Is Machine Learning?

Machine Learning is a method of creating AI systems that learn patterns from data instead of receiving every rule directly from a programmer.

In traditional programming, we provide:

Rules + Data → Output

For example, a programmer may write rules to identify whether an email is spam.

In machine learning, we provide:

Data + Correct Examples → Learned Model

The model studies the examples and learns patterns. Later, it uses those patterns to make predictions about new data. Machine learning is commonly described as learning patterns from training data and using them to make inferences about unseen data. [ibm](#)

Example: Spam detection

Suppose we give a machine learning system thousands of emails:

Email	Correct answer
"You won a free prize!"	Spam
"Meeting at 10 AM tomorrow"	Not spam
"Claim your cash reward now!"	Spam

The system may discover patterns such as:

- Certain words frequently appear in spam.
- Spam emails may contain many links.
- Unknown senders may be more common in spam.
- Excessive capital letters may be suspicious.

The programmer does not need to write every possible spam rule. The algorithm learns useful patterns from examples.

Machine learning is not magic

A machine learning model does not understand information in exactly the same way humans do. It identifies mathematical relationships in data.

If the training data is incomplete, incorrect, or biased, the model may also produce poor or unfair results.

3. What Is Deep Learning?

Deep Learning is a branch of machine learning that uses neural networks with multiple layers.

The word “deep” refers to the presence of several processing layers between the input and the output. These layers allow the system to learn increasingly complex patterns.

For example, when recognizing a face in an image:

Pixels → Edges → Shapes → Facial parts → Complete face

A deep learning model may learn these levels automatically.

Machine learning versus deep learning

Feature	Traditional machine learning	Deep learning
Main technique	Many types of algorithms	Multi-layer neural networks
Data requirement	Often works with smaller datasets	Usually benefits from very large datasets
Feature design	Humans often select useful features	The model can learn features automatically
Hardware	Can often run on ordinary computers	Often benefits from GPUs or specialized hardware
Typical examples	Decision trees, linear regression, support vector machines	Image recognition, speech recognition, large language models

Consider house-price prediction.

With traditional machine learning, a human might select:

- Number of bedrooms.
- Area of the house.
- Location.
- Age of the building.

With deep learning, the system may learn useful representations directly from more complex inputs, such as photographs, text descriptions, maps, and numerical data.

Important relationship

Deep learning is not separate from machine learning. It is one particular approach inside machine learning:

AI: The overall goal

ML: Learning patterns from data

DL: Using many-layered neural networks to learn patterns

4. The Basic AI/ML Process

Most machine learning systems follow a general process:

Collect data



Prepare data



Train a model



Evaluate the model



Use the model for predictions

Let us examine each part.

4.1 Data

Data is the information from which the system learns.

Data can include:

- Numbers.
- Text.
- Images.
- Audio.
- Video.
- Sensor readings.
- Customer records.
- Medical measurements.

For a model that predicts student exam results, the data might contain:

- Hours studied.
- Attendance percentage.
- Previous marks.
- Assignment scores.
- Final examination result.

Each individual item is often called a **data point** or **example**.

4.2 Features

A **feature** is an individual property or input used by a model.

For a house-price model:

`Features = area, number of rooms, location, age`

For an email model:

`Features = words, sender, number of links, message length`

Features are the information given to the model so that it can produce an answer.

4.3 Labels

A **label** is the correct answer associated with an example.

For email classification:

Input: "Claim your free prize"

Label: Spam

For image classification:

Input: Image of a dog

Label: Dog

Labels are especially important in **supervised learning**, where the system learns from examples that already have correct answers.

4.4 Training data

Training data is the portion of data used to teach the model.

The model examines many examples and adjusts its internal values so that its answers become more accurate.

For example:

Training example:

Input: 5 hours studied

Correct output: 78 marks

After seeing many examples, the model may learn that students who study more hours often achieve higher marks. This does not mean studying time is the only factor, but it may be one useful pattern.

4.5 The model

A **model** is the learned mathematical system that connects inputs to outputs.

Before training, the model does not yet know the useful relationships in the data. During training, it changes internal parameters until it can make reasonably accurate predictions.

You can think of a model as a student:

- Training data is the textbook and practice exercises.
- The learning algorithm is the study method.
- The model is the student's learned knowledge.
- A prediction is the student's answer to a new question.

4.6 Prediction

A **prediction** is the output produced by a trained model.

The prediction may be:

- A category: "cat" or "dog."
- A number: house price of ₹75 lakh.
- A probability: 90% chance of spam.
- A sentence or paragraph.
- A recommendation.
- A decision.

The word "prediction" does not always mean guessing the future. It can also mean identifying or generating an answer from current input.

5. How Training Works

Training usually follows a repeated cycle:

1. Give input to the model.
2. The model produces a prediction.
3. Compare the prediction with the correct answer.
4. Measure the error.
5. Adjust the model.
6. Repeat many times.

Example

Suppose a model is learning to recognize cats.

Input: Cat image

Model prediction: Dog

Correct answer: Cat

The model has made an error. A mathematical function called a **loss function** measures how wrong the prediction was.

The learning process then adjusts the model's internal parameters to reduce future errors. This prediction–error–adjustment cycle is central to training neural networks.[databricks](https://www.databricks.com)

After seeing many examples, the model may gradually learn:

- Fur patterns.
- Ear shapes.
- Eye positions.
- Body shapes.
- Typical cat outlines.

Loss

Loss is a numerical measurement of how incorrect a prediction is.

For example:

Small loss = prediction is close to the correct answer

Large loss = prediction is far from the correct answer

The goal during training is generally to reduce the loss.

Parameters

Parameters are internal values that the model learns during training.

In a neural network, these parameters are primarily called **weights** and **biases**.

They determine how strongly different inputs affect the final output.

6. Training, Validation, and Testing

A dataset is commonly divided into separate parts.

Training set

The model uses this data to learn patterns.

Validation set

This data helps developers choose settings and check whether the model is improving during development.

Test set

This data is kept separate until the end. It provides an estimate of how well the model performs on new, unseen examples.

A simple division might look like:

Training data: 70%

Validation data: 15%

Testing data: 15%

The exact percentages may vary.

Why not test on training data?

Imagine a student memorizes every question in a practice book. If the examination contains exactly the same questions, the student may score well without truly understanding the subject.

A model can do something similar. It may memorize the training examples instead of learning general patterns.

This problem is called **overfitting**.

Overfitting

Overfitting occurs when a model performs very well on training data but poorly on new data.

Training performance: Excellent

New-data performance: Poor

A good model should **generalize**, meaning it should perform well on examples it has not seen before.

Underfitting

Underfitting occurs when the model is too simple or has not learned enough.

Training performance: Poor

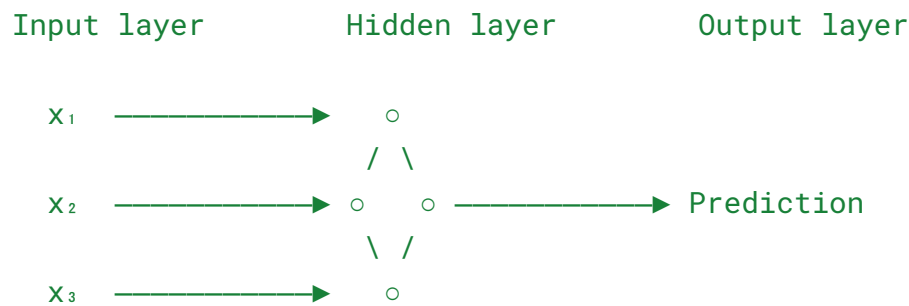
New-data performance: Poor

The goal is to find a useful balance between underfitting and overfitting.

7. Neural Network Architecture

A neural network is a machine learning model made of connected processing units called **nodes**, arranged into **layers**.

A simplified neural network looks like this:



The actual networks used in modern AI may contain thousands, millions, or billions of connections.

7.1 Nodes

A **node**, also called a neuron, receives numbers, performs a calculation, and passes a result to the next layer.

A node does not work exactly like a biological brain cell. The name is an analogy. In practice, it performs mathematical operations.

A simple node:

1. Receives input values.
2. Multiplies each input by a weight.
3. Adds the results together.
4. Adds a bias.
5. Applies an activation function.
6. Sends the output forward.

In simplified mathematical form:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Then the node applies an activation function:

output=f(z)

You do not need to calculate this by hand initially. The important idea is that a node combines information and decides how strongly to pass it forward.

8. The Main Layers

Input layer

The **input layer** receives the original information.

Examples:

- For a house-price model: area, rooms, and location.
- For an image model: pixel values.
- For a speech model: sound measurements.
- For a text model: numerical representations of words or tokens.

The input layer does not usually make a decision. It simply receives the data.

Hidden layers

The **hidden layers** process the information.

They are called hidden because their internal calculations are not directly visible as the final answer.

Different layers may learn different levels of patterns.

For an image:

First layer: edges

Second layer: corners and textures

Third layer: eyes, ears, or wheels

Later layer: complete object

For text, different layers may learn relationships involving:

- Words.
- Grammar.
- Sentence structure.
- Meaning.
- Context.

Output layer

The **output layer** produces the final result.

Examples:

- One output value for a predicted house price.
- Two outputs for “cat” or “dog.”
- Several outputs for multiple categories.
- A probability distribution over possible next words.

The design of the output layer depends on the task.

9. Weights and Biases

Weights

A **weight** represents the importance of an input connection.

Suppose a model predicts whether a student will pass:

Attendance — weight

Study hours — weight

Previous marks — weight

If study hours are especially useful, the model may learn a relatively strong weight for that input. If an input is less useful, its weight may become smaller.

Weights are not manually chosen one by one. The training process adjusts them automatically.

Biases

A **bias** is an additional adjustable value that helps a node shift its output.

Without a bias, a node’s behavior may be too restricted. The bias gives the node additional flexibility when learning a pattern.

A useful analogy is:

- Weight: How important is this piece of information?
- Bias: What starting adjustment should be applied?

Learning means adjusting parameters

When we say that a neural network “learns,” we usually mean that it adjusts its weights and biases to reduce prediction errors.

Initial weights → Predictions with errors

Adjusted weights → Better predictions

Repeated adjustment → Learned model

10. Activation Functions

An **activation function** determines how strongly a node should respond to its input.

Without activation functions, stacking many layers would behave too much like one simple mathematical operation. Activation functions allow networks to learn more complicated relationships.

You may encounter names such as:

- ReLU.
- Sigmoid.
- Tanh.
- Softmax.

At an introductory level, remember:

An activation function helps a neural network decide what information should continue forward and how strongly it should continue.

For classification tasks, an output layer may use a function that converts results into probabilities.

Example:

Cat: 0.85

Dog: 0.10

Rabbit: 0.05

The model would usually select “cat” as the highest-probability class.

11. A Complete Example

Imagine building a model that identifies whether a fruit is an apple or an orange.

Step 1: Collect data

For many fruits, record:

- Weight.
- Color.
- Size.
- Shape.

Also record the correct label:

Apple or Orange

Step 2: Provide input

A new fruit is described as:

Weight: 150 grams

Color: red

Size: medium

Shape: round

Step 3: Process the input

The input layer receives these values. Hidden layers combine them using weights and activation functions.

Step 4: Produce a prediction

The output layer might produce:

Apple: 0.92

Orange: 0.08

The model predicts “apple.”

Step 5: Compare with the correct answer

If the fruit really is an apple, the loss is small. If it is actually an orange, the model receives a larger error and adjusts its parameters during training.

Step 6: Repeat

The model studies many fruits and improves its ability to classify new ones.

12. Important Beginner Terms

Term	Simple meaning
AI	The broad field of making machines perform intelligent tasks
Machine learning	Learning patterns from data to make predictions or decisions
Deep learning	Machine learning using neural networks with many layers
Data	Information used by a system
Feature	An input property used for prediction
Label	The correct answer attached to an example
Model	The learned system that maps inputs to outputs
Training	The process of adjusting a model using examples
Prediction	The output produced by a model
Parameter	An internal value learned by the model
Weight	A parameter indicating the importance of a connection
Bias	A parameter that shifts a node's behavior
Loss	A measurement of prediction error
Neural network	Connected layers of mathematical processing units
Generalization	Performing well on new, unseen data
Overfitting	Memorizing training data instead of learning general patterns

13. Final Mental Model

Remember this story:

AI is the overall idea of making machines intelligent. Machine learning is a way to achieve AI by allowing machines to learn patterns from data. Deep learning is a type of machine learning that uses many-layered neural networks. The network receives inputs, processes them through nodes and layers using weights and biases, and produces a prediction. During training, it compares predictions with correct answers and adjusts its parameters to reduce errors.

The complete picture is:

Real-world data



Features and labels



Training algorithm



Model learns weights and biases



New input



Prediction



Evaluation and improvement

Large Language Models

A **Large Language Model**, or LLM, is an AI model trained on a very large collection of text. Its basic task is to predict what token should come next, but its huge size, training data, and neural-network architecture allow it to perform tasks such as answering questions, summarizing, translating, writing code, and solving many structured problems.

The central idea is:

Text → Tokens → Neural network → Probabilities for next token →
Generated response

Modern LLMs are usually based on the **Transformer** architecture, whose main innovation is the attention mechanism. Attention allows the model to consider how different words relate to one another, even when they are far apart in a sentence.

1. What Is an LLM?

An LLM is a neural-network model trained to understand and generate language. It learns from large amounts of text such as:

- Books.
- Articles.
- Websites.
- Documentation.
- Conversations.
- Source code.
- Other text collections.

The model does not store language as a simple dictionary of sentences. Instead, training adjusts billions of numerical parameters so that the model learns patterns involving:

- Vocabulary.
- Grammar.
- Facts.
- Writing styles.
- Relationships between concepts.
- Common reasoning patterns.
- Code structures.

The basic task: predict the next token

Suppose the model receives:

The sun rises in the

It calculates possible next tokens:

east: 0.96
west: 0.01
morning: 0.01
sky: 0.01
other: 0.01

It then selects or samples one token, such as:

east

The process continues:

The sun rises in the east

The model predicts another token, and this repeats until it produces a complete answer.

What is a token?

A **token** is a small unit of text processed by an LLM.

A token may be:

- A complete word.
- Part of a word.
- A punctuation mark.
- A space combined with a word.

For example, a sentence might be divided approximately as:

Artificial intelligence is useful.

into tokens such as:

"Artificial" | " intelligence" | " is" | " useful" | "."

The exact tokenization depends on the model.

LLMs do not directly process ordinary words as humans see them. They convert text into tokens, and tokens into numerical vectors called embeddings.

2. How Next-Token Prediction Becomes Powerful

At first, “predict the next word” may sound simple. However, predicting the next token accurately requires understanding a great deal of context.

Consider:

The trophy would not fit in the suitcase because it was too large.

What does “it” refer to?

Most people understand that the trophy was too large. To predict the next words correctly, the model must recognize relationships between:

- “it.”
- “trophy.”
- “suitcase.”
- “too large.”

Now consider a programming example:

```
javascript
function add(a, b) {
  return
```

A language model trained on code may predict:

```
javascript
a + b;
```

To do this well, it must recognize syntax and the purpose of the function.

Repeated prediction creates text generation

An LLM generates text one token at a time:

Input: Explain photosynthesis in one sentence.

Token 1: Photosynthesis

Token 2: is

Token 3: the

Token 4: process

...

Every newly generated token becomes part of the context for predicting the following token.

Previous context → Predict next token → Add token → Predict again

Why this can look like reasoning

During training, the model encounters many examples of:

- Explanations.
- Arguments.
- Proofs.
- Calculations.
- Procedures.
- Questions and answers.
- Programs and documentation.

As a result, it can learn patterns that resemble reasoning.

For example, given:

All birds have wings.

A sparrow is a bird.

Therefore, a sparrow has

the likely continuation is:

wings.

The model is producing a continuation based on learned patterns. It may appear to reason, but it can still make mistakes, invent information, or fail on problems that require reliable verification.

Important limitation

An LLM does not automatically guarantee that every answer is true. It generates likely text, not guaranteed facts.

Therefore:

- Check important factual claims.
- Do not treat confident wording as proof.
- Test generated code.
- Use reliable sources for medical, legal, financial, and safety decisions.

3. A Simplified LLM Pipeline

A typical language-model pipeline can be understood as:

```
User text
↓
Tokenization
↓
Token IDs
↓
Embeddings
↓
Transformer layers
↓
Probability distribution
↓
Selected next token
↓
Decoded text
```

Tokenization

The input sentence is divided into tokens.

"Learning AI is exciting"

might become something similar to:

```
["Learning", " AI", " is", " exciting"]
```

Each token is converted into an integer ID.

Embeddings

The token ID is mapped to a vector of numbers.

For example:

"cat" → [0.21, -0.44, 0.73, ...]

These numbers do not have an easy human interpretation. They provide a mathematical representation that the neural network can process.

Transformer processing

The embeddings pass through many Transformer layers. Each layer updates the representation of each token by considering other relevant tokens in the context.

Output probabilities

The final layer produces scores for possible next tokens. A function called **softmax** converts those scores into probabilities.

"east" → 0.82
"west" → 0.04
"sky" → 0.03
"morning" → 0.02

The system then chooses a token according to its generation settings.

4. The Transformer Architecture

Before Transformers became popular, many language systems used recurrent neural networks, or RNNs, which processed text sequentially:

Word 1 → Word 2 → Word 3 → Word 4

This made it difficult to process very long passages efficiently and to preserve relationships over long distances.

The Transformer introduced **self-attention**, allowing tokens in a sequence to examine other tokens directly. This also makes much more parallel processing possible during training.[arxiv](#)

A simplified Transformer block contains:

Input representations



Self-attention



Feed-forward network



Output representations

Large language models commonly stack many such blocks.

Main components

- Token embeddings.
- Positional information.
- Self-attention.
- Multi-head attention.
- Feed-forward networks.
- Residual connections.
- Layer normalization.
- Output projection.

You do not need to memorize every formula at the beginning. Focus first on the role of attention.

5. Attention: The Core Idea

Attention allows each token to decide which other tokens are important for understanding its current context.

Consider:

The student dropped the glass because it was fragile.

When interpreting “it,” the model should pay more attention to “glass” than to “student.”

Attention creates a set of importance values:

"it" → glass: high attention
"it" → student: lower attention
"it" → the: very low attention

These values help produce a context-aware representation of the token "it."

An everyday analogy

Imagine reading this sentence:

The dog chased the ball because it was moving.

To understand "it," you look back at the sentence and decide which earlier word matters. You give more mental attention to "ball" than to "the."

Self-attention performs a mathematical version of this process for every token.

6. Queries, Keys, and Values

Attention uses three useful concepts:

- **Query:** What information is this token looking for?
- **Key:** What kind of information does each token contain?
- **Value:** What actual information should be passed along?

Suppose the current token is "it."

Query: I need to identify what "it" refers to.

Keys: "student", "dropped", "glass", "fragile"

Values: The information associated with each token.

The query is compared with the keys. If a key seems relevant, that token receives a higher attention score. The model then combines the corresponding values.

The standard simplified attention formula is:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

You can understand the formula in four steps:

1. Compare queries with keys.
2. Calculate relevance scores.
3. Convert scores into weights using softmax.
4. Combine the values using those weights.

The result is a representation that includes relevant context, rather than treating each token independently.

A numerical illustration

Suppose the token “it” assigns these attention weights:

```
student: 0.10  
glass:   0.75  
fragile: 0.10  
other:   0.05
```

The model gives the largest influence to the information associated with “glass.” These are illustrative values, not the exact internal values of a real model.

7. Self-Attention and Causal Attention

Self-attention

In self-attention, tokens attend to other tokens in the same sequence.

For example, every token in:

```
The cat sat on the mat.
```

can use information from other tokens to build a better contextual representation.

Causal attention

When generating text from left to right, the model must not look at future tokens that it is supposed to predict.

For example, when predicting:

```
The cat sat
```

the model may use:

The
The cat
The cat sat

but it must not use the future word:

The cat sat on the mat

A **causal mask** hides future positions during generation.

This ensures the model learns the correct prediction task:

Past and current tokens → Next token

8. Multi-Head Attention

A Transformer does not use only one attention calculation. It uses several attention “heads” in parallel.

Different heads may focus on different relationships:

- One head may track grammatical relationships.
- Another may connect a pronoun to a noun.
- Another may notice nearby phrases.
- Another may follow a programming variable.
- Another may identify important topic words.

The results from all heads are combined.

Input

|— Attention head 1
|— Attention head 2
|— Attention head 3
|— Attention head 4

↓

Combined representation

This is called **multi-head self-attention**.

The advantage is that the model can examine several types of relationships at the same time.

9. What Happens Inside a Transformer Block?

A simplified Transformer block works like this:

Token representations

↓

Add positional information

↓

Self-attention finds relevant context

↓

Residual connection and normalization

↓

Feed-forward network transforms each position

↓

Residual connection and normalization

↓

Next Transformer block

Feed-forward network

After attention mixes information across tokens, a feed-forward network processes each token representation independently through additional layers.

A simple explanation:

- Attention decides which other tokens matter.
- The feed-forward network transforms and refines the information.

Residual connections

A residual connection allows information from an earlier stage to be added to a later stage.

This helps very deep networks train effectively and preserve useful information.

Layer normalization

Normalization keeps numerical values in a manageable range, helping training remain stable.

10. Base Models and Instruction-Tuned Models

LLMs are often discussed in two broad forms: **base models** and **instruction-tuned models**.

Base model

A base model is trained mainly to predict the next token in text.

It learns language patterns from large datasets, but it may not reliably behave like an assistant.

If you ask:

Explain gravity to a beginner.

a base model might continue with:

Explain gravity to a beginner. Gravity is...

It may complete the text rather than clearly follow your request.

A base model is like a highly capable language-completion engine.

Instruction-tuned model

An instruction-tuned model receives additional training on examples such as:

Instruction: Explain gravity to a beginner.

Response: Gravity is the force that attracts objects toward one another...

This teaches it to:

- Follow commands.
- Answer questions.
- Use requested formats.
- Hold conversations.
- Refuse some unsafe requests.
- Produce more useful assistant-style responses.

Instruction tuning is also called supervised fine-tuning when the model learns from carefully prepared instruction–response pairs.

Comparison

Feature	Base model	Instruction-tuned model
Main training goal	Predict continuation of text	Follow instructions and produce useful responses
Typical behavior	Completes text	Answers or performs a requested task
Interaction style	Less predictable	More conversational
Best use	Further training, research, specialized completion	Chatbots, assistants, applications
Example prompt	Text continuation	Direct question or instruction

An instruction-tuned model is still a next-token prediction model. The additional training changes the kinds of continuations it has learned to produce.

11. Prompt Engineering

Prompt engineering is the practice of writing clear instructions and context so that an LLM is more likely to produce the desired result.

A prompt is not just a question. It may include:

- The role of the model.
- The task.
- Relevant background.
- Constraints.
- Examples.
- Output format.
- The target audience.

OpenAI's prompt-engineering guidance recommends making the task, tone, goals, and examples clear; it also distinguishes higher-level instructions from the user's input.

Weak prompt

Tell me about databases.

This is vague. The model does not know:

- The audience.
- The length.
- The purpose.
- The desired level of detail.
- Whether examples are needed.

Better prompt

You are teaching first-year college students.

Explain relational databases in simple language. Include:

1. A definition.
2. The meaning of tables, rows, and columns.
3. One example using a student database.
4. Three beginner mistakes to avoid.

Use headings and keep the answer under 500 words.

This prompt provides:

- Role.
- Audience.
- Topic.
- Required sections.
- Example context.
- Length constraint.
- Output format.

A useful prompt structure

Role:

You are a patient programming instructor.

Task:

Explain JavaScript promises.

Context:

The learner understands variables and functions but not asynchronous programming.

Requirements:

- Use simple language.
- Include one analogy.
- Provide one short code example.
- Explain the output.
- Avoid advanced terminology unless defined.

Format:

Use headings and bullet points.

Be specific about the output

Instead of:

`Summarize this.`

try:

`Summarize the text in five bullet points. Each bullet must contain one main idea and no more than 20 words.`

Give examples

Examples help communicate the pattern you want:

`Convert the following records into JSON.`

Example:

`Input: Name: Asha, Age: 20`

`Output:`

`{"name": "Asha", "age": 20}`

`Now convert:`

`Name: Ravi, Age: 21`

This is called **few-shot prompting** because the prompt gives a small number of examples.

Separate instructions from content

Use clear delimiters:

Instruction:

Summarize the text in three points.

Text to summarize:

Artificial intelligence is...

This makes it easier for the model to distinguish what it must do from the information it must process.

Ask for structured output

For application development, structure is often better than free-form prose:

Return valid JSON with these fields:

```
{
  "topic": "string",
  "difficulty": "beginner | intermediate | advanced",
  "summary": "string",
  "examples": ["string"]
}
```

Your application should still validate the result rather than blindly trusting it.

Prompt engineering is iterative

A practical process is:

1. Write a simple prompt.
2. Test it with several inputs.
3. Identify incorrect or inconsistent outputs.
4. Add missing context or constraints.
5. Test again.
6. Save the improved prompt and examples.

Prompt engineering is not about finding a magical phrase. It is about clearly specifying the task and evaluating the result.

12. Making an API Call with Node.js

An API allows your program to communicate with an LLM service.

The general flow is:

Your Node.js program



HTTP/API request



LLM provider



Generated response



Your application

The official OpenAI JavaScript/TypeScript SDK can be installed with npm and used from server-side JavaScript environments such as Node.js.

Step 1: Create a project

```
bash
mkdir llm-demo
cd llm-demo
npm init -y
npm install openai
```

You can use JavaScript or TypeScript. The following example uses modern JavaScript modules.

Step 2: Store the API key securely

Set your API key as an environment variable.

On macOS or Linux:

```
bash
export OPENAI_API_KEY="your_api_key_here"
```

On Windows PowerShell:

```
powershell
$env:OPENAI_API_KEY="your_api_key_here"
```

Do not place secret API keys directly in code or commit them to GitHub.

Step 3: Create a basic request

Create `example.mjs`:

```
javascript
import OpenAI from "openai";

const client = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});

const response = await client.responses.create({
  model: "gpt-5.6",
  input: "Explain recursion to a beginner in three sentences.",
});

console.log(response.output_text);
```

Run it:

```
bash
node example.mjs
```

The current official quickstart uses the JavaScript SDK, `client.responses.create(...)`, a model identifier, and `response.output_text` to read the generated text. Model names can change, so use a model currently available to your account and provider. [developers.openai](https://developers.openai.com)

What each part means

```
javascript
import OpenAI from "openai";
```

Loads the SDK.

```
javascript
const client = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});
```

Creates a client using the secret API key.

```
javascript
await client.responses.create(...)
```

Sends a request to generate a response.

```
javascript
model: "gpt-5.6"
```

Selects the model. In real projects, choose a currently supported model according to your provider's documentation.

```
javascript
input: "Explain recursion..."
```

Provides the instruction or user message.

```
javascript
response.output_text
```

Reads the generated text.

13. A TypeScript Example

Install TypeScript tools:

```
bash
npm install openai
npm install -D typescript tsx @types/node
```

Create `example.ts`:

```
typescript
import OpenAI from "openai";
```

```
const client = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});

async function main(): Promise<void> {
  const response = await client.responses.create({
    model: "gpt-5.6",
    instructions:
      "You are a patient teacher. Explain technical ideas for
beginners.",
    input:
      "Explain the difference between an array and an object in
JavaScript. Include one example of each.",
  });

  console.log(response.output_text);
}

main().catch((error: unknown) => {
  console.error("Request failed:", error);
  process.exitCode = 1;
});
```

Run it with:

```
bash
npx tsx example.ts
```

The `instructions` field is useful for high-level behavior such as role, tone, and response expectations, while `input` contains the actual task. The provider's prompt guidance states that instructions can take priority over the input prompt.

14. A Practical Prompt in Code

typescript

```
const topic = "neural networks";

const response = await client.responses.create({
  model: "gpt-5.6",
  instructions: `
You are an introductory AI professor.
Use plain language and define every technical term.
Do not assume the student knows calculus or programming.
`,
  input: `
Teach the topic: ${topic}

Return:
1. A one-sentence definition.
2. A simple real-world analogy.
3. The three most important components.
4. One short example.
5. Three review questions.
`,
});

console.log(response.output_text);
```

This design separates:

- Stable behavior in `instructions`.
- The changing topic and task in `input`.

That makes your program easier to maintain.

15. Streaming Responses

Normally, your program waits for the complete response. With **streaming**, the model sends portions of the answer as they become available.

This is useful for chat interfaces because users can see the answer appearing gradually. Streaming responses use server-sent events, and the JavaScript SDK can process text-delta events as they arrive.

typescript

```
const stream = await client.responses.create({
  model: "gpt-5.6",
  input: "Explain the water cycle in simple language.",
  stream: true,
});

for await (const event of stream) {
  if (event.type === "response.output_text.delta") {
    process.stdout.write(event.delta);
  }
}
```

Conceptually:

Request sent

↓

First text fragment arrives

↓

More fragments arrive

↓

Complete response

16. API Safety and Good Practice

When working with LLM APIs:

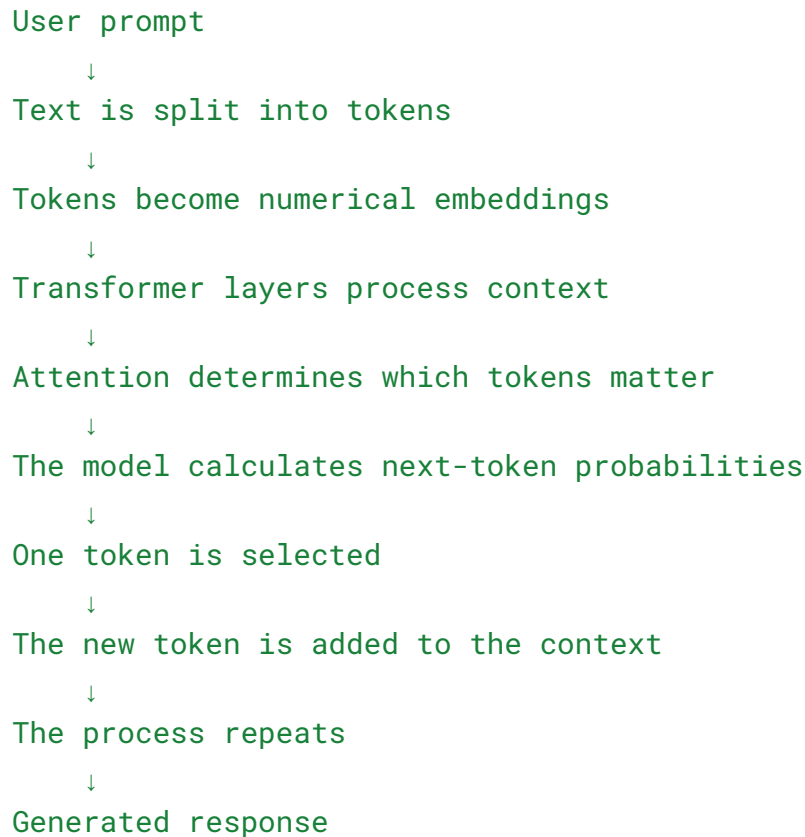
- Keep API keys on the server, never in browser code.
- Store keys in environment variables or a secure secret manager.
- Validate model output before using it in your application.
- Set reasonable limits on input and output length.
- Handle network failures and rate limits.
- Do not assume generated text is factually correct.
- Avoid sending private or sensitive information unless the service and your policy allow it.
- Log carefully; logs can accidentally expose user data.
- Test prompts with many different inputs.

- Use structured output when your application needs predictable data.

A generated answer is an untrusted external result until your application checks it.

17. The Complete Mental Model

You can now connect all the concepts:

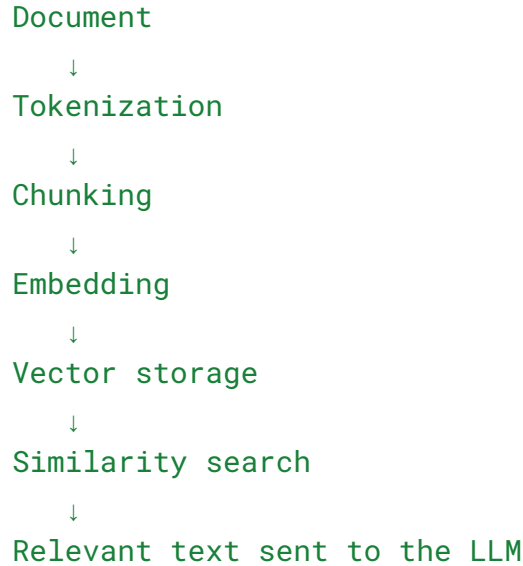


The most important points are:

1. An LLM generates text by predicting one token at a time.
2. Its apparent intelligence comes from learning complex language and reasoning patterns during large-scale training.
3. Transformers make this possible through self-attention.
4. Attention lets tokens use relevant information from other tokens in the context.
5. Base models mainly complete text, while instruction-tuned models are trained to follow requests.
6. Good prompts specify the role, task, context, constraints, and output format.
7. A Node.js or TypeScript application can call an LLM through an SDK and use the generated response in a larger system.

Data Processing for AI

When an AI system works with a large document, it usually does not send the entire document directly to an LLM. Instead, it prepares the document through several steps:



These steps are especially important in systems such as semantic search and **Retrieval-Augmented Generation**, or RAG.

1. Why Process Documents?

Suppose you have a 500-page college handbook and ask:

What is the attendance requirement for final examinations?

Sending all 500 pages every time would be inefficient because:

- The document may be larger than the model's context limit.
- Processing the whole document costs more.
- Most pages are irrelevant to the question.
- Searching exact keywords may miss related wording.

Instead, the system:

1. Splits the document into smaller pieces.
2. Converts each piece into a numerical representation.
3. Finds the pieces most related to the question.
4. Sends only those pieces to the LLM.

Question → Search relevant chunks → Give chunks to LLM → Generate answer

2. Tokenization

What is tokenization?

Tokenization is the process of breaking text into smaller units called tokens.

LLMs do not directly read text as humans do. They convert text into tokens, and then convert those tokens into numerical IDs that the neural network can process. A token may be a complete word, part of a word, punctuation, or a space combined with a word. [developers.openai](https://openai.com/blog/openai-api)

For example, this sentence:

Artificial intelligence is useful.

might be represented approximately as:

["Artificial", " intelligence", " is", " useful", "."]

The exact result depends on the tokenizer.

Words are not always tokens

A short common word may be one token:

the

A less common or longer word may be divided:

text

tokenization

→ token + ization

This is why token count is not exactly the same as word count. As a rough English estimate, one token is approximately four characters or 0.75 words, although this varies by language and text type.

Why tokenization matters

Tokenization affects:

- How much text fits into a model's context window.
- API cost.
- Processing speed.
- Chunk size.
- Whether a document exceeds an embedding model's limit.
- How much text can be included in a prompt.

For example, a document containing 10,000 words may contain more than 10,000 tokens because some words are split into multiple tokens.

Token IDs

After splitting text, the tokenizer converts each token into an integer ID:

```
"AI"           → 1542
" is"          → 318
" useful"      → 7643
```

These numbers are identifiers, not meaningful quantities. Token ID 7643 is not “more important” than token ID 318.

The model then converts these IDs into embeddings that its neural network can process.

Tokenization pipeline

Original text

↓

Normalize text

↓

Split into tokens

↓

Convert tokens to IDs

↓

Send IDs to the model

Tokenizers usually handle normalization, splitting, adding special tokens, converting tokens to IDs, and decoding IDs back into text.

3. Text Chunking

What is chunking?

Text chunking is the process of dividing a large document into smaller, meaningful pieces.

For example:

Large document

↓

Chunk 1

Chunk 2

Chunk 3

Chunk 4

Each chunk can then be embedded, stored, searched, and sent to an LLM separately.

Why not split every 500 characters?

A simple split may break important ideas:

Chunk 1: Students must submit their application before...

Chunk 2: ...the final deadline.

The second chunk is incomplete and difficult to understand by itself.

Good chunking attempts to preserve meaning.

4. Chunking Strategies

4.1 Fixed-size chunking

The document is divided into chunks of a fixed number of tokens or characters.

Example:

Chunk size: 500 tokens

Overlap: 50 tokens

Advantages:

- Simple to implement.
- Predictable chunk sizes.
- Easy to scale.

Disadvantages:

- May cut sentences or ideas in the middle.
- Does not understand document structure.

A practical starting point for general documents might be a few hundred tokens with some overlap, but the best size depends on the document and search task. Microsoft's chunking guidance describes fixed-size chunks with overlap as a useful starting strategy and gives 512 tokens with 25% overlap as one example—not as a universal rule.

4.2 Sentence-based chunking

The document is split at sentence boundaries.

Sentence 1

Sentence 2

Sentence 3

Advantages:

- Avoids cutting sentences in half.
- Produces more readable chunks.

Disadvantages:

- A fixed number of sentences may be too short or too long.
- A single sentence may not contain enough context.

4.3 Paragraph-based chunking

The system uses paragraph boundaries.

Paragraph 1 → Chunk 1
Paragraph 2 → Chunk 2
Paragraph 3 → Chunk 3

This often works well for ordinary articles and notes because paragraphs generally contain a complete idea.

4.4 Heading-based chunking

Documents such as Markdown, HTML, and manuals often have headings:

3. Attendance Policy
3.1 Minimum Attendance
3.2 Medical Exceptions

A heading-aware splitter can preserve the relationship between a section and its subsection.

For example:

Section: Attendance Policy
Subsection: Medical Exceptions
Content: Students may apply for...

This is often better than splitting only by character count because the heading provides useful context. Document structure such as headings, paragraphs, page boundaries, and HTML markup can be used to create more meaningful chunks.

4.5 Semantic chunking

Semantic chunking groups sentences that discuss the same idea.

For example, these sentences may belong together:

The university allows students to retake one examination.
The retake must be requested within seven days.
The examination office publishes the retake schedule.

Even if the paragraph boundaries are unusual, these sentences form one meaningful topic.

Semantic chunking can produce better retrieval results, but it is more complex and may require another model to measure semantic similarity.

4.6 Document-specific chunking

Different data types need different strategies:

Document type	Useful approach
Legal contract	Clause- and section-based chunks
Technical manual	Heading- and subsection-based chunks
Research paper	Section, paragraph, and equation-aware chunks
Source code	Function- or class-based chunks
Spreadsheet	Row-, column-, or table-based chunks
Chat history	Conversation-turn chunks
Web page	Heading and paragraph chunks
PDF	Page and heading-aware chunks

There is no single chunking method that is best for every document.

5. Chunk Overlap

What is overlap?

Overlap means repeating a small part of one chunk in the next chunk.

Without overlap:

Chunk 1: A B C D E
Chunk 2: F G H I J

With overlap:

Chunk 1: A B C D E
Chunk 2: D E F G H

The repeated content helps preserve context when an important idea crosses a chunk boundary.

Example

Original text:

Students must submit the form before the deadline. Late applications may be rejected unless the student provides approved documentation.

Poor split:

Chunk 1: Students must submit the form before the deadline. Late applications

Chunk 2: may be rejected unless the student provides approved documentation.

With overlap:

Chunk 1: Students must submit the form before the deadline. Late applications

Chunk 2: Late applications may be rejected unless the student provides approved documentation.

Advantages of overlap

- Preserves boundary context.
- Reduces incomplete ideas.
- Improves the chance that a search result contains the full answer.

Disadvantages of overlap

- Increases storage.
- Increases embedding cost.
- May produce duplicate search results.
- Does not fix poorly chosen chunk boundaries.

A common starting point is 10–25% overlap, but this should be evaluated using real questions and retrieval results.

6. Chunk Metadata

A chunk should not be stored as text alone. It should also carry useful metadata.

Example:

```
json
{
  "chunkId": "handbook-042",
  "documentId": "student-handbook-2026",
  "text": "Students must maintain at least 75% attendance...",
  "section": "Attendance Policy",
  "page": 18,
  "startToken": 8200,
  "endToken": 8700
}
```

Metadata helps with:

- Showing the source page.
- Filtering by document type.
- Restricting searches to a department.
- Displaying citations.
- Updating or deleting a document.
- Reconstructing the original document.
- Debugging retrieval problems.

A useful rule is:

Store the chunk, its embedding, and enough metadata to explain where the chunk came from.

7. Embeddings

What is an embedding?

An **embedding** is a **vector representation of data, such as text, designed to preserve useful aspects of its content or meaning**. Texts with similar meanings tend to have embeddings that are closer together in vector space.

For example:

Text A: How do I reset my password?

Text B: I forgot my password. How can I create a new one?

These sentences use different words but have similar meanings. Their embeddings should be relatively close.

By contrast:

Text C: How do I bake a chocolate cake?

would likely be farther away.

Important distinction

An embedding model usually does not generate an answer. It converts text into numbers that can be used for:

- Semantic search.
- Similarity comparison.
- Document clustering.
- Recommendation systems.
- Duplicate detection.
- Classification.
- Retrieval-Augmented Generation.

Embedding models take text as input and produce embedding vectors as output.

8. How Text Becomes an Embedding

Suppose we have this chunk:

Students must maintain at least 75% attendance to sit for the final examination.

The processing looks like:

Text chunk
↓
Tokens
↓
Neural network
↓
Meaning-related numerical representation
↓
Embedding vector

A simplified vector might look like:

[0.12, -0.44, 0.87, 0.03, -0.19, ...]

Real embedding vectors usually contain many more dimensions.

The individual numbers do not usually have simple labels such as:

0.12 = attendance
-0.44 = examination

Meaning is distributed across the vector as a pattern of values.

9. Vectors

What is a vector?

A vector is an ordered list of numbers.

Example:

`[0.12, -0.44, 0.87, 0.03]`

In machine learning, vectors are used to represent data mathematically.

For embeddings:

`Text → [number, number, number, ...]`

The number of values is called the vector's **dimension** or **dimensionality**.

For example:

`Vector A = [0.2, 0.7, -0.1]`

`Dimension = 3`

An embedding model may produce vectors with hundreds or thousands of dimensions.

Why many dimensions?

Language meaning is complex. A vector needs enough numerical space to represent distinctions involving:

- Topics.
- Relationships.
- Style.
- Intent.
- Entities.
- Context.
- Similarity.

A higher-dimensional vector can represent complex patterns, but it also requires more storage and computation.

10. Similarity Between Vectors

Once text is represented as vectors, we can compare vectors mathematically.

The most common comparison method is **cosine similarity**.

It measures the angle between two vectors rather than simply comparing their lengths.

cosine similarity(A,B)= $\frac{A \cdot B}{\|A\| \|B\|}$ $\text{cosine similarity}(A,B) = \frac{A \cdot B}{\|A\| \|B\|}$

Where:

- $A \cdot B$ is the dot product.
- $\|A\|$ is the length of vector A.
- $\|B\|$ is the length of vector B.

A simplified interpretation:

Similarity near 1 → very similar direction

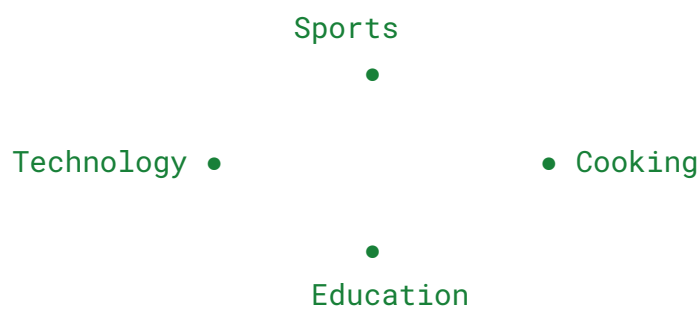
Similarity near 0 → weak similarity

Similarity near -1 → opposite direction

The exact useful range depends on the embedding model and the data.

Two-dimensional analogy

Imagine a map:



In a real embedding system, the map may have hundreds or thousands of dimensions, not just two. But the idea is similar: items with related meanings tend to be near one another.

11. Semantic Search

Traditional keyword search looks for matching words.

Query:

How can I change my password?

Keyword search looks for terms such as:

change, password

Semantic search uses embeddings to find related meaning.

It may also retrieve:

To update your login credentials, open Account Settings and select Reset Password.

Even though the wording is different, the meaning is related.

Semantic search process

1. Split documents into chunks.
2. Create an embedding for every chunk.
3. Store the chunks and vectors.
4. Embed the user's question.
5. Compare the question vector with stored vectors.
6. Return the closest chunks.

Question → Query embedding → Similarity search → Relevant chunks

12. Embeddings in RAG

In a Retrieval-Augmented Generation system, embeddings help find useful information before the LLM writes an answer.

Indexing stage

This usually happens before the user asks a question:

Documents

↓

Clean text

↓

Create chunks

↓

Create embeddings

↓

Store vectors and metadata

Query stage

This happens when a user asks a question:

User question

↓

Create query embedding

↓

Search for similar chunk vectors

↓

Retrieve top chunks

↓

Place chunks into the LLM prompt

↓

Generate answer

Example:

Question:

What attendance is needed to take the final examination?

Retrieved context:

Students must maintain at least 75% attendance to sit for the final examination.

LLM answer:

Students need at least 75% attendance to sit for the final examination.

The LLM is responsible for generating the response. The embedding system is mainly responsible for finding relevant information.

13. Simple Node.js Embedding Example

The following example shows the general shape of an embedding request using the OpenAI JavaScript SDK. The embedding endpoint accepts text and an embedding model, then returns an embedding vector.

bash

```
npm install openai
```

```
javascript
```

```
import OpenAI from "openai";
```

```
const client = new OpenAI({  
  apiKey: process.env.OPENAI_API_KEY,  
});
```

```
const text = `  
Students must maintain at least 75% attendance  
to sit for the final examination.  
`;
```

```
const result = await client.embeddings.create({  
  model: "text-embedding-3-small",  
  input: text,  
});
```

```
const vector = result.data[0].embedding;
```

```
console.log("Vector dimensions:", vector.length);  
console.log("First five values:", vector.slice(0, 5));
```

The output may look like:

```
Vector dimensions: 1536
```

```
First five values: [0.012, -0.044, 0.087, 0.003, -0.019]
```

The exact dimensions and model availability depend on the selected embedding model and provider.

Embedding multiple chunks

```
javascript
```

```
const chunks = [  
  "Students must maintain at least 75% attendance.",  
  "Applications for leave must be submitted to the department  
office.",  
  "The final examination schedule is published two weeks in advance.",  
];
```

```
const result = await client.embeddings.create({
  model: "text-embedding-3-small",
  input: chunks,
});

const vectors = result.data.map((item) => item.embedding);

console.log("Number of vectors:", vectors.length);
```

Each input chunk receives its own vector.

14. A Simple Chunking Example

This is a basic paragraph-based approach:

javascript

```
function chunkText(text, maxCharacters = 1000, overlap = 100) {
  const chunks = [];
  let start = 0;

  while (start < text.length) {
    const end = Math.min(start + maxCharacters, text.length);
    const chunk = text.slice(start, end).trim();

    if (chunk) {
      chunks.push(chunk);
    }

    if (end === text.length) {
      break;
    }

    start = end - overlap;
  }

  return chunks;
}
```

```
const documentText = `
Long document content goes here...
`;

const chunks = chunkText(documentText);

console.log(chunks);
```

This demonstrates the idea, but production systems should prefer better boundaries such as:

- Headings.
- Paragraphs.
- Sentences.
- Code functions.
- Table rows.
- Page sections.

Also, production systems should measure chunk sizes in tokens rather than relying only on characters, because model limits are token-based.

15. Choosing a Good Chunk Size

There is no universal perfect chunk size.

A chunk that is too small may lose context:

75% attendance

A chunk that is too large may contain several unrelated topics:

Attendance rules + library policy + examination timetable + hostel rules

A useful chunk should be:

- Large enough to express a complete idea.
- Small enough to retrieve precisely.
- Within the embedding model's input limit.
- Appropriate for the question-answering task.

General starting points

For ordinary text, you might initially test:

Chunk size: 300–700 tokens

Overlap: 10–20%

For technical manuals, heading-based chunks may be more useful than a fixed number. For code, split by functions or classes. For tables, preserve headers with each group of rows.

These are starting points, not permanent rules. Evaluate them using real user questions.

16. Common Problems

Problem 1: Chunks are too small

Symptoms:

- Answers lack context.
- Pronouns such as “this” or “they” become unclear.
- Search returns fragments.

Improvement:

- Increase chunk size.
- Add overlap.
- Include the section heading in each chunk.

Problem 2: Chunks are too large

Symptoms:

- Search returns broad, noisy sections.
- The relevant answer is buried inside a large chunk.
- More tokens are sent to the LLM.

Improvement:

- Split by headings or paragraphs.
- Reduce chunk size.
- Remove unrelated content.

Problem 3: Poor document extraction

Symptoms:

- PDF columns appear in the wrong order.
- Tables become unreadable.
- Headers and footers are repeated.
- Page numbers interfere with search.

Improvement:

- Clean and inspect extracted text before chunking.
- Use document-type-specific extraction.
- Preserve table structure where possible.

Problem 4: Missing metadata

Symptoms:

- The system finds an answer but cannot show the source.
- Updating one document is difficult.
- Search results cannot be filtered.

Improvement:

- Store document ID, page, section, offsets, and timestamps.

Problem 5: Similarity is not enough

A mathematically similar chunk is not always the correct answer. Embeddings can retrieve relevant-looking text that is incomplete or outdated.

Improvement:

- Retrieve several candidates.
- Use metadata filters.
- Add keyword or hybrid search.
- Rerank results.
- Ask the LLM to answer only from retrieved context.
- Display sources.

17. Important Distinctions

Concept	Main purpose
Tokenization	Converts text into model-readable pieces
Chunking	Divides large content into manageable meaningful sections
Embedding	Converts text meaning into a numerical representation
Vector	The numerical array produced by an embedding model
Vector database	Stores vectors and supports similarity search
LLM	Generates or analyzes language
RAG	Retrieves relevant information and gives it to an LLM

A compact example:

```
"Attendance must be 75%"
  ↓ tokenization
["Attendance", " must", " be", " 75%"]
  ↓ embedding
[0.12, -0.44, 0.87, ...]
  ↓ vector database
Compared with the question vector
  ↓ retrieval
Relevant chunk sent to the LLM
```

18. Final Mental Model

Remember the process this way:

- **Tokenization** answers: “How does the model divide text?”
- **Chunking** answers: “How do we divide a large document into useful pieces?”
- **Embeddings** answer: “How do we represent the meaning of each piece numerically?”
- **Vectors** are: “The arrays of numbers used for comparison.”
- **Similarity search** answers: “Which stored pieces are most related to this question?”

The full pipeline is:

Raw document



Clean and extract text



Tokenize and count tokens



Create meaningful chunks



Generate one embedding per chunk



Store vectors with metadata



Embed the user's question



Find similar vectors



Retrieve relevant chunks



Send them to the LLM



Generate a grounded answer

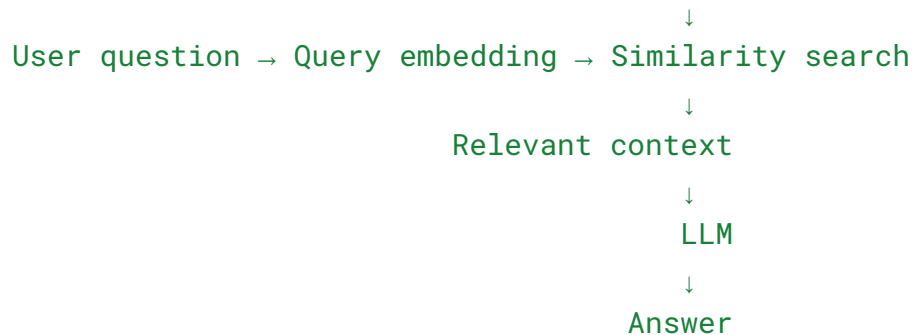
The central idea is simple:

Convert language into numbers that preserve useful relationships, so a computer can find text with similar meaning even when the exact words are different.

Vector Storage, RAG, and AI Agents

These topics connect the earlier ideas into complete AI applications:

Documents → Chunks → Embeddings → Vector database



When we add tools and actions, the system becomes more agent-like:

User request → LLM plans → Calls a tool → Receives result



1. Vector Databases

What is a vector database?

A **vector database** stores numerical vectors and allows us to search for vectors that are similar to a query vector.

From the previous topic:

Text chunk → Embedding model → Vector

A vector database stores something like:

```
{
  "text": "Students must maintain 75% attendance.",
  "embedding": [0.12, -0.44, 0.87, 0.03],
  "metadata": {
    "document": "student-handbook.pdf",
    "page": 18,
    "section": "Attendance"
  }
}
```

The vector is used for similarity search. The text and metadata are returned so the application knows what the vector represents.

Why use a vector database?

A normal database is excellent for queries such as:

```
sql
SELECT * FROM students WHERE department = 'Computer Science';
```

A vector database is designed for queries such as:

```
Find the document chunks whose meanings are most similar to:
"How much attendance do I need for exams?"
```

The words do not need to match exactly. The system searches by semantic similarity.

What does a vector database store?

A typical record contains:

- The original text chunk.
- Its embedding vector.
- Document ID.
- Page number.
- Section title.
- Access permissions.
- Creation or update time.

- Other application metadata.

A vector without its source text is usually not useful by itself. Always store enough metadata to explain and retrieve the original content.

2. PostgreSQL and pgvector

You do not always need a separate vector database. If your application already uses PostgreSQL, the **pgvector** extension allows PostgreSQL to store vectors and perform similarity searches.

pgvector supports exact and approximate nearest-neighbor search, along with distance measures such as Euclidean distance, inner product, and cosine distance.

This is useful because you can keep ordinary application data and vector data together.

Traditional PostgreSQL

PostgreSQL

- └─ students
- └─ courses
- └─ payments
- └─ documents

PostgreSQL with pgvector

PostgreSQL + pgvector

- └─ students
- └─ courses
- └─ payments
- └─ documents
 - └─ document_chunks
 - └─ embedding vector

This lets you combine normal filters with semantic search:

Find chunks similar to the question
AND only from the Computer Science department
AND only from documents updated this year

Example database table

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```
CREATE TABLE document_chunks (  
    id BIGSERIAL PRIMARY KEY,  
    document_id TEXT NOT NULL,  
    content TEXT NOT NULL,  
    page_number INTEGER,  
    section_title TEXT,  
    embedding VECTOR(1536)  
);
```

The number `1536` is only an example. It must match the dimensionality of the embedding model you use.

Insert a chunk

```
INSERT INTO document_chunks  
    (document_id, content, page_number, section_title, embedding)  
VALUES  
    (  
        'student-handbook-2026',  
        'Students must maintain at least 75% attendance.',  
        18,  
        'Attendance Policy',  
        '[0.12, -0.44, 0.87, ...]'  
    );
```

In real application code, the vector is generated by an embedding model rather than typed manually.

3. Vector Similarity Search

What is similarity search?

Vector similarity search finds stored vectors that are closest to a query vector.

The process is:

User question



Create question embedding



Compare it with stored chunk embeddings



Return the nearest chunks

The returned chunks are often called **nearest neighbors**.

Cosine similarity

Cosine similarity compares the angle between two vectors:

$$\text{cosine similarity}(A,B) = \frac{A \cdot B}{\|A\| \|B\|}$$

$\text{cosine similarity}(A,B) = \frac{A \cdot B}{\|A\| \|B\|}$

Here:

- $A \cdot B$ is the dot product.
- $\|A\|$ is the length of vector A.
- $\|B\|$ is the length of vector B.

The basic idea is:

Similar direction → Similar meaning

Different direction → Less similar meaning

The vector's direction is usually more important than its absolute size for semantic comparison.

Small example

Suppose the query is:

What attendance percentage is required?

The database might return:

Rank	Retrieved chunk	Similarity
1	Students need 75% attendance to take final examinations.	0.91
2	Attendance records are published every month.	0.74
3	Library membership must be renewed annually.	0.18

The first chunk is the most relevant.

The numbers are illustrative. Real scores depend on the embedding model and data.

Cosine distance in pgvector

pgvector uses the `<=>` operator for cosine distance. Cosine similarity can be calculated as:

```
1 - (embedding <=> query_embedding)
```

The pgvector documentation gives the same relationship: cosine similarity is one minus cosine distance.

Example:

```
SELECT
  id,
  content,
  page_number,
  section_title,
  1 - (embedding <=> $1::vector) AS similarity
FROM document_chunks
ORDER BY embedding <=> $1::vector
LIMIT 5;
```

Here:

- `$1` is the query embedding.
- The database orders rows by closest cosine distance.
- The five most relevant chunks are returned.

Similarity search is not truth verification

A high similarity score means:

This chunk is mathematically close to the question.

It does not guarantee:

This chunk completely and correctly answers the question.

A good system should also consider:

- Document authority.
- Document date.
- Access permissions.
- Completeness.
- Conflicting sources.
- Keyword matches.
- Whether the answer is actually present in the chunk.

4. Dedicated Vector Databases

A dedicated vector database may be useful when:

- Vector search is the central purpose of the application.
- You have very large collections of embeddings.
- You need specialized indexing or scaling.
- You need built-in filtering and hybrid search.
- Your data is already distributed across services.

Examples include:

- Pinecone.
- Weaviate.
- Qdrant.
- Milvus.
- Chroma.
- Elasticsearch vector search.
- PostgreSQL with pgvector.

The best choice depends on:

- Dataset size.
- Hosting requirements.
- Cost.
- Existing infrastructure.
- Search latency.
- Metadata filtering.
- Team familiarity.

For a beginner project, PostgreSQL plus pgvector is often a practical starting point if you already know PostgreSQL. It avoids immediately learning a completely separate database system.

5. What Is RAG?

Retrieval-Augmented Generation, or **RAG**, combines information retrieval with text generation.

Instead of asking an LLM to answer using only its learned parameters, the application first retrieves relevant information and places it into the prompt.

Retrieval + Generation = RAG

The retrieved content gives the LLM current or private information, such as:

- Company policies.
- Product documentation.
- Internal reports.
- University regulations.
- Customer records.
- Legal documents.
- Technical manuals.

LlamaIndex describes RAG as indexing data and selectively retrieving relevant parts to provide to an LLM as source material for a response.

6. The RAG Architecture

A RAG system has two major stages.

Stage A: Indexing

Indexing is performed before users ask questions.

Documents



Extract text



Clean text



Create chunks



Generate embeddings



Store chunks, vectors, and metadata
student-handbook.pdf



Attendance section



Chunk 42



Embedding vector



PostgreSQL + pgvector

Stage B: Retrieval and generation

This happens when a user asks a question.

User question



Create query embedding



Search vector database



Retrieve top relevant chunks



Build a prompt with question + context



Send prompt to LLM



Generate answer

Complete workflow

User:

"What attendance is required for final exams?"



Embedding model:

Converts question to a vector



Vector database:

Finds similar handbook chunks



Retrieved context:

"Students need at least 75% attendance..."



LLM:

Uses the question and context



Answer:

"Students need at least 75% attendance..."

7. RAG Prompt Design

A RAG application usually creates a prompt like this:

You are a helpful university assistant.

Answer the question using only the context below.

If the answer is not present in the context, say that you do not have enough information. Do not invent a policy.

Context:

Students must maintain at least 75% attendance to sit for the final examination.

Question:

What attendance is required for final examinations?

This prompt has three important parts:

- The assistant's behavior.
- The retrieved context.
- The user's question.

Why tell the LLM to use context only?

Without a clear instruction, the LLM may combine retrieved information with unrelated knowledge or make an unsupported guess.

A more reliable RAG system should:

- Tell the LLM to use the retrieved sources.
- Tell it what to do when context is insufficient.
- Include source metadata.
- Return citations or document references.
- Validate sensitive answers.

RAG reduces dependence on the model's internal knowledge, but it does not guarantee perfect answers. Bad extraction, poor chunks, or irrelevant retrieval can still lead to incorrect responses.

8. Basic RAG in TypeScript

The following is a simplified conceptual example.

```
type RetrievedChunk = {
  content: string;
  pageNumber?: number;
  sectionTitle?: string;
};

async function answerQuestion(question: string): Promise<string> {
  const queryEmbedding = await createEmbedding(question);

  const chunks: RetrievedChunk[] =
```

```

    await searchVectorDatabase(queryEmbedding, 5);

    const context = chunks
      .map((chunk, index) => {
        return `[Source ${index + 1}]
${chunk.content}`;
      })
      .join("\n\n");

    const prompt = `
Answer the question using only the provided context.
If the context does not contain the answer, say so clearly.

Context:
${context}

Question:
${question}
`;

    return await askLLM(prompt);
  }
}

```

The important separation is:

Embedding model → Finds relevant chunks
 LLM → Writes the final answer

The embedding model and the generation model perform different jobs.

9. Frameworks for RAG

Building every part manually is possible, but frameworks can reduce repetitive code.

LangChain.js

LangChain.js provides building blocks for:

- Document loading.

- Text splitting.
- Embedding models.
- Vector stores.
- Retrievers.
- Prompt templates.
- LLM calls.
- Tool calling.
- Agent workflows.

Its JavaScript documentation describes vector stores that can be converted into retrievers and used in question-answering chains.

A typical LangChain-style pipeline is:

Loader → Splitter → Embeddings → Vector store

↓

Question → Retriever → Prompt → LLM

LlamaIndex.TS

LlamaIndex.TS is a TypeScript/JavaScript framework focused on connecting private or external data to LLM applications.

Its RAG documentation describes indexing data, storing vector embeddings, retrieving relevant source material, and giving that material to an LLM.

It can help with:

- Document ingestion.
- Node or chunk creation.
- Vector indexes.
- Retrievers.
- Query engines.
- Agents.
- Persistent vector stores.

Frameworks versus custom code

Approach	Benefits	Trade-offs
Custom implementation	Maximum control and easier learning	More code to maintain
LangChain.js	Many reusable components and integrations	Abstraction can feel complex
LlamaIndex.TS	Strong focus on data and RAG workflows	Requires learning its concepts
PostgreSQL + pgvector	Uses familiar database infrastructure	You manage more of the pipeline
Dedicated vector database	Specialized search and scaling	Adds another infrastructure component

A good learning path is:

1. Build a small RAG system manually.
2. Understand each step.
3. Use a framework to improve productivity.
4. Inspect what the framework is doing.

10. What Is an AI Agent?

An AI agent is a system in which an LLM helps decide what steps to take, which tools to use, and how to respond to the result.

A chatbot usually follows:

Question → Answer

An agent may follow:

```

Request
  ↓
Plan
  ↓
Choose a tool
  ↓
Execute tool

```

↓
Observe result
↓
Choose the next step
↓
Final answer
Find my unpaid invoices and email me a summary.

An agent might:

1. Search the billing database.
2. Filter unpaid invoices.
3. Calculate the total.
4. Draft a summary.
5. Send an email after approval.

The LLM itself should not automatically have unrestricted access to your systems. Your application controls the available tools and executes them.

11. Tool Calling and Function Calling

What is tool calling?

Tool calling allows an LLM to request that your application execute a predefined function.

The model does not directly execute your database query or API request. Instead:

1. Your application describes available tools to the model.
2. The model requests a tool call.
3. Your application validates the arguments.
4. Your application executes the function.
5. Your application sends the result back to the model.
6. The model produces a final answer or requests another tool.

This is the standard multi-step tool-calling flow described in the OpenAI documentation.

Example tool

Suppose your application exposes:

```
type SearchDocumentsArgs = {  
  query: string;  
  department?: string;  
};
```

The tool description might say:

```
search_documents:  
Search university documents for relevant information.  
Arguments:  
- query: the user's search question  
- department: optional department filter
```

The model may request:

```
{  
  "name": "search_documents",  
  "arguments": {  
    "query": "What is the attendance requirement for final exams?",  
    "department": "Computer Science"  
  }  
}
```

Your server then:

- Validates the arguments.
- Creates the embedding.
- Queries PostgreSQL.
- Returns matching chunks.

The model receives the result:

```
{  
  "results": [  
    {  
      "content": "Students must maintain at least 75% attendance...",  
      "page": 18  
    }  
  ]  
}
```

The model can then write:

Computer Science students need at least 75% attendance...

Important distinction

The LLM chooses or requests the tool, but your backend performs the actual operation.

LLM: "Please call search_documents."

Backend: Validates and executes search_documents.

LLM: Uses the returned result.

This separation is essential for security.

12. Calling an Express.js Endpoint

Suppose you have an Express server:

```
import express from "express";

const app = express();

app.use(express.json());

app.get("/api/weather", async (req, res) => {
  const city = String(req.query.city);

  const weather = await getWeatherForCity(city);

  res.json(weather);
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Your agent application can define a tool that calls this endpoint:

```
async function getWeather(city: string) {
```

```
const response = await fetch(
  `http://localhost:3000/api/weather?city=${encodeURIComponent(city)}`
);

if (!response.ok) {
  throw new Error(`Weather request failed: ${response.status}`);
}

return response.json();
}
```

The model might request:

```
{
  "name": "get_weather",
  "arguments": {
    "city": "Bengaluru"
  }
}
```

Your application executes:

```
const result = await getWeather("Bengaluru");
```

Then it returns the result to the LLM.

13. Tool-Calling Safety

Tool calling can cause real side effects, so every tool should have protections.

Read-only tools

Examples:

- Search documents.
- Get weather.
- Look up an order.
- Query a product catalog.

These are usually lower risk, but they still need authentication and input validation.

Side-effect tools

Examples:

- Send an email.
- Delete a record.
- Transfer money.
- Update a database.
- Cancel an order.
- Create a calendar event.

These need stronger controls:

- Require user confirmation.
- Validate permissions.
- Check arguments.
- Limit accessible records.
- Log the operation.
- Make operations idempotent where possible.
- Use human approval for sensitive actions.

Never allow a model to generate arbitrary SQL and execute it without strict controls.

14. ReAct: Reasoning and Acting

ReAct means **Reasoning and Acting**.

The pattern interleaves thinking about the task with actions that interact with the outside world:

Thought → Action → Observation → Thought → Action → Observation

The ReAct approach combines reasoning traces with tool actions and environmental feedback so the system can plan, gather information, and adjust its approach.

Example

What is the current stock status of product X, and email me if it is unavailable?

Thought:

I need to check the inventory first.

Action:

```
check_inventory(product = "X")
```

Observation:

Product X is unavailable.

Thought:

The user asked to be emailed if unavailable. I need confirmation before sending.

Action:

```
ask_for_confirmation()
```

Observation:

User approved the email.

Action:

```
send_email(...)
```

The agent does not need to perform every step in one fixed chain. It can use the result of one tool call to decide the next step.

Why ReAct is useful

It helps with tasks that require:

- External information.
- Multiple steps.
- Conditional decisions.
- Recovery from errors.
- Combining tools.
- Checking intermediate results.

Do not expose private reasoning unnecessarily

In production systems, it is often better to show:

I'm checking the inventory and will notify you if the item is unavailable.

rather than exposing private internal reasoning. Applications should log concise action steps and tool results for auditing.

15. Workflow Versus Agent

Not every AI application needs an agent.

Fixed workflow

Use a fixed workflow when the steps are known:

```
Upload document
  ↓
Extract text
  ↓
Create chunks
  ↓
Generate embeddings
  ↓
Store vectors
```

This is predictable and easier to test.

Agent workflow

Use an agent when the system must decide among tools or steps:

```
User request
  ↓
LLM decides whether to search documents
  ↓
Maybe call a database tool
  ↓
Maybe call a calculator
  ↓
Maybe ask for confirmation
  ↓
Respond
```

A fixed workflow is often safer for routine tasks. An agent is useful when flexibility is genuinely needed.

16. Memory Management

An LLM does not automatically remember every past conversation forever. Your application must decide what information to store and what to send with each request.

There are two main types of memory.

Short-term conversational memory

Short-term memory is the current conversation history.

User: My name is Asha.

Assistant: Nice to meet you, Asha.

User: What is my name?

To answer the final question, the application may include the previous messages in the current request.

Example:

```
const messages = [
  {
    role: "user",
    content: "My name is Asha.",
  },
  {
    role: "assistant",
    content: "Nice to meet you, Asha.",
  },
  {
    role: "user",
    content: "What is my name?",
  },
];
```

Short-term memory problems

Conversation history can become:

- Too long.
- Expensive.
- Irrelevant.
- Conflicting.
- Above the model's context limit.

Common solutions include:

- Keep only recent messages.
- Summarize older messages.
- Retrieve relevant previous turns.
- Remove repeated information.
- Store important facts separately.

Long-term memory

Long-term memory stores information that may be useful across sessions.

Examples:

- User preference for concise answers.
- Preferred programming language.
- Account settings.
- Previously approved project details.
- Important facts explicitly saved by the user.

Long-term memory should not save everything automatically. It needs:

- Clear storage rules.
- User control.
- Privacy protection.
- Update and deletion support.
- Access control.
- Expiration policies.

Memory using a database

A memory record might contain:

json

```
{  
  "userId": "user-123",
```

```
{
  "memory": "User prefers TypeScript examples.",
  "category": "preference",
  "confidence": 0.95,
  "createdAt": "2026-09-23T10:00:00Z",
  "expiresAt": null
}
```

The application can embed the memory and retrieve relevant memories when needed.

Current question

↓

Create query embedding

↓

Search memory vectors

↓

Retrieve relevant preferences

↓

Add them to the prompt

This is similar to RAG, except the retrieved information is about the user or previous work rather than general documents.

17. Agent Memory Design

A practical agent may use several memory layers:

Current turn

↓

Recent conversation history

↓

Conversation summary

↓

Retrieved long-term memories

↓

External knowledge base

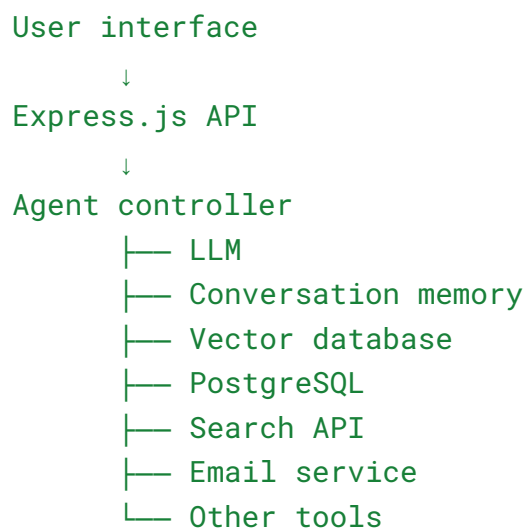
Each layer serves a different purpose:

Memory type	Purpose
Current input	Understand the immediate request
Recent messages	Maintain conversational continuity
Summary	Preserve older conversation context compactly
Long-term memory	Store durable preferences or facts
Vector knowledge base	Retrieve external documents
Application database	Store authoritative records

Do not use an LLM memory store as the source of truth for payments, permissions, inventory, or legal records. Use a proper database for authoritative data.

18. Complete Agent Architecture

A realistic application may look like this:



The agent controller is responsible for:

- Managing conversation state.
- Sending prompts to the LLM.
- Handling tool calls.
- Validating arguments.

- Executing tools.
- Returning tool results.
- Requesting confirmation.
- Limiting loops.
- Logging actions.
- Handling failures.

The LLM should not directly control your infrastructure. It should operate within carefully designed application boundaries.

19. Complete Example: University Assistant

Can I take the final exam, and email my advisor if I cannot?

1. Identify the student.
2. Search the university policy documents.
3. Check the student's attendance record.
4. Compare attendance with the policy.
5. Explain the result.
6. If an email is needed, draft it.
7. Ask the student for confirmation.
8. Send the email only after approval.

The tools might be:

```
search_policy_documents()
get_student_attendance()
draft_email()
send_email()
```

The agent should not send the email immediately after deciding that it is needed. Sending an email is an external side effect and should require confirmation.

20. Final Mental Model

The complete system can be remembered in four layers.

Layer 1: Knowledge preparation

Documents → Chunks → Embeddings → Vector database

Layer 2: Retrieval

Question → Query embedding → Similarity search → Relevant chunks

Layer 3: Generation

Question + Retrieved chunks → LLM → Answer

Layer 4: Action

LLM decision → Tool call → Backend execution → Result → LLM

And memory fits across these layers:

Conversation history

User preferences

Retrieved documents

Database records

Tool results

The most important principles are:

- Vector databases store and search semantic representations.
- Cosine similarity estimates how closely two vectors point in the same direction.
- pgvector lets PostgreSQL perform vector search without immediately adopting a separate database.
- RAG retrieves relevant information before asking the LLM to answer.
- Frameworks such as LangChain.js and LlamaIndex.TS provide reusable components for these pipelines.
- Agents use LLMs to choose steps and tools, but the backend must execute and control those tools.
- ReAct alternates between reasoning, acting, and observing results.
- Short-term memory maintains the current conversation; long-term memory stores selected durable information.
- Sensitive actions require validation, authorization, and usually explicit user confirmation.